UNIT V • PYTHON

Data Structures & NumPy

Teaching slides for first-year students



Lists • Tuples • Dictionaries • Sets • Numerical Arrays

Learning roadmap

We will cover the syllabus up to NumPy in this presentation.

Why data structures matter

Lists: indexing, slicing, methods and traversal

Tuples: immutability and unpacking

Dictionaries: key-value pairs and frequency counting

Sets: uniqueness and mathematical operations

NumPy: arrays, shape, arithmetic and mean

Bridge to sensor-data analysis

Learning outcome

Choose a suitable data structure and perform basic numerical analysis using NumPy.

1. What is a data structure?

A way to organize and store data so it can be accessed and processed.

```
mark1 = 78
mark2 = 85
mark3 = 91

marks = [78, 85, 91]
print(marks)
```

Why use collections?

Keep related values together, reduce separate variables and make iteration easier.

Four structures

List • Tuple • Dictionary • Set

2. Lists: the basics

A list is an ordered, mutable collection.

```
marks = [78, 85, 91, 67, 88]
names = ["Aman", "Riya", "Rahul"]
mixed = [10, "Python", 3.14, True]
empty = []
print(marks)
```

List properties

Ordered • Mutable • Duplicates allowed • Mixed data types allowed

Syntax

Square brackets [ ]; elements separated by commas.

Lists: indexing

Python uses zero-based indexing.

```
numbers = [10, 20, 30, 40, 50]
print(numbers[0])   # 10
print(numbers[2])   # 30
print(numbers[-1])  # 50
print(numbers[-2])  # 40
```

Remember

First element: index 0
Last element: index –1
Invalid index → IndexError



Lists: slicing

Syntax: list_name[start:stop:step] — stop is excluded.

```
a = [10, 20, 30, 40, 50, 60]
print(a[1:4])  # [20, 30, 40]
print(a[:3])   # [10, 20, 30]
print(a[2:])   # [30, 40, 50, 60]
print(a[::2])  # [10, 30, 50]
print(a[::-1]) # reverse list
```

Three rules

start is included
stop is excluded
step controls the interval

Lists: updating and adding

Lists are mutable, so elements can be changed after creation.

```
a = [10, 20, 30]
a[1] = 25
a.append(40)
a.extend([50, 60])
a.insert(1, 15)
print(a)
```

append vs extend

append([3, 4]) adds one nested list.
extend([3, 4]) adds two elements.

Methods

append(x) • extend(iterable) • insert(i, x)

Lists: removing and useful methods

remove(x): removes first matching value

pop() / pop(i): removes and returns an element

clear(): removes all elements

len(a): number of elements

sum(a), min(a), max(a): numerical operations

count(x), index(x): count / find first index

sort() changes the list; sorted(a) returns a new list

Exam tip

remove() raises ValueError if absent. pop() raises IndexError if the list or index is invalid.

Lists: traversal and a practical program

Visit each element with a loop.

```
numbers = [23, 56, 12, 89, 45]
largest = numbers[0]
for num in numbers:
 if num > largest:
  largest = num
  print("Largest:", largest)  # 89
```

Algorithm

1. Assume first value is largest.
2. Compare every value.
3. Update when a larger value appears.
4. Print the result.

Complexity

O(n): each element is checked once.

3. Tuples

A tuple is an ordered collection whose elements cannot be reassigned.

```
coordinates = (10, 20)
student = ("Riya", 18, 85.5)
one_item = (10,)
print(student[0])  # Riya
print(student[-1]) # 85.5
print(student[1:]) # (18, 85.5)
```

Important

(10,) is a tuple; (10) is an integer. The comma matters.

Typical uses

Coordinates, fixed records and RGB colours

Tuples: immutability and unpacking

```
student = ("Riya", 18, 90)
name, age, marks = student
print(name, age, marks)
a, b = 10, 20
a, b = b, a
print(a, b)  # 20 10
```

List vs tuple

List: [1, 2, 3] — mutable\nTuple: (1, 2, 3) — immutable

Subtle point

A tuple may contain a mutable object, such as a list; that inner list can still change.

4. Dictionaries

A dictionary stores data as key-value pairs.

```
student = {
    "name": "Aman",
    "age": 18,
    "marks": 85
}
print(student["name"])  # Aman
print(student["marks"]) # 85
```

Think of it as

'name' → 'Aman'
'age' → 18
'marks' → 85

Properties

Mutable • Unique keys • Insertion order preserved in modern Python

Dictionaries: update, retrieve and traverse

```
student["age"] = 19       # update
student["grade"] = "A"    # add
print(student.get("grade"))
for key, value in student.items():
 print(key, ":", value)
```

Useful methods

keys() • values() • items()
get() • update() • pop()

[] vs get()

student['x'] raises KeyError if missing.
student.get('x') returns None by default.

Dictionary application: frequency counting

```
text = "banana"
frequency = {}
for ch in text:
  frequency[ch] = frequency.get(ch, 0) + 1
print(frequency)
# {"b": 1, "a": 3, "n": 2}
```

How it works

For each character: read count (or 0), add one, and store the new count.

Applications

Character counts, word frequencies, votes and event totals

5. Sets

A set stores unique elements and supports mathematical set operations.

```
numbers = {10, 20, 30, 20, 10}
print(numbers)  # {10, 20, 30}
a = set([1, 2, 2, 3])
empty_set = set()
empty_dict = {}
```

Remember

Sets remove duplicates, do not support indexing, and have no guaranteed display order.

Empty set

Use set(), not {}. Empty braces create a dictionary.

Sets: mathematical operations

Let A = {1, 2, 3, 4} and B = {3, 4, 5, 6}.

Union A | B: all elements → {1, 2, 3, 4, 5, 6}

Intersection A & B: common elements → {3, 4}

Difference A - B: in A but not B → {1, 2}

Symmetric difference A ^ B: in one set, not both → {1, 2, 5, 6}

Equivalent methods: union(), intersection(), difference()

Practical use

Remove duplicates or compare groups of students, courses or IDs.

Choosing the right data structure

Structure	Syntax	Mutable?	Best for	Example
List	[ ]	Yes	Changing sequence	[1, 2, 3]
Tuple	( )	No	Fixed values	(x, y)
Dictionary	{key: value}	Yes	Key-value mapping	{"name": "Riya"}
Set	{ } / set()	Yes	Unique elements	{1, 2, 3}

Quick choice

Changing sequence → list | Fixed values → tuple | Lookup by key → dictionary | Unique items → set

6. Introduction to NumPy

NumPy = Numerical Python; a library for numerical computation.

Creates numerical arrays and performs operations on them

Useful in data science, scientific computing, AI and sensor analysis

Install if needed: pip install numpy

Import convention: import numpy as np

The big idea

NumPy can apply an operation to a whole array without an explicit loop.

Why use NumPy arrays?

Lists and arrays behave differently during arithmetic.

```
marks = [70, 80, 90]
print(marks * 2)
# [70, 80, 90, 70, 80, 90]
marks_np = np.array([70, 80, 90])
print(marks_np * 2)
# [140 160 180]
```

Core distinction

List * 2 repeats the list.
NumPy array * 2 multiplies each element.

Import first

import numpy as np

Difference between List and numpy array

Basis	Python List	NumPy Array
1. Purpose	General-purpose data storage	Numerical and scientific computing
2. Syntax	`a = [1, 2, 3]`	`a = np.array([1, 2, 3])`
3. Arithmetic	`a * 2` repeats the list	`a * 2` multiplies every element
4. Adding a number	`a + 2` gives an error	`a + 2` adds 2 to every element
5. Data types	Can easily store mixed types	Usually stores elements in a common data type
6. Dimensions	Can represent tables using nested lists	Supports multidimensional arrays with numerical operations
7. Performance	Often less efficient for large numerical calculations	Generally faster for large numerical calculations

Creating arrays and understanding shape

```
import numpy as np
a = np.array([10, 20, 30, 40])
print(a.ndim)   # 1
print(a.shape)  # (4,)
print(a.size)   # 4

b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)  # (2, 3)
```

Three properties

ndim = number of dimensions
shape = size along each axis
size = total number of elements

2D example

Shape (2, 3) = 2 rows × 3 columns = 6 values.

NumPy indexing

Use row index first, then column index for a 2D array.

```
a = np.array([[10, 20, 30],
              [40, 50, 60]])
print(a[0, 1])  # 20
print(a[1, 2])  # 60
print(a[0])     # first row
```

Index reminder

Indexing starts at 0.
a[0, 1] means row 0, column 1.

NumPy: arithmetic and scalar operations

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
print(a + b)  # [11 22 33]
print(a - b)  # [ 9 18 27]
print(a * b)  # [10 40 90]
print(a / b)  # [10. 10. 10.]
print(a + 5)  # [15 25 35]
```

Element-wise operations

Corresponding elements are combined. Arrays need compatible shapes.

Scalar

A scalar is one value, such as 5. NumPy applies it to each element.

NumPy: calculating the mean

Mean = sum of values ÷ number of values.

```
temperatures = np.array([28, 30, 32, 29, 31])
average = np.mean(temperatures)
print("Average:", average)
# Average: 30.0
```

Calculation

(28 + 30 + 32 + 29 + 31) ÷ 5 = 30

Related functions

np.sum(a) • np.min(a) • np.max(a)\nFocus on np.mean() and basic arithmetic.

Mini application: analyse sensor readings

A temperature sensor records values over time.

```
readings = np.array([27.5, 28.0, 29.5, 30.0, 28.5])
print("Readings:", readings)
print("Average:", np.mean(readings))
print("Highest:", np.max(readings))
print("Lowest:", np.min(readings))
```

Why this matters

Sensor readings become numerical data that can be summarized and later plotted.

Next step

Matplotlib will turn readings into a line graph.

Unit recap: what you should know

List: ordered, mutable, supports indexing and slicing

Tuple: ordered, immutable; supports unpacking

Dictionary: mutable key-value mapping; keys are unique

Set: unique elements; union, intersection and difference

NumPy: arrays, dimensions, shape, arithmetic and mean

Next: file handling, CSV, Matplotlib and IoT demo

Exit question

Choose a structure for changing marks, a fixed coordinate, student ID → name, and unique course IDs.

Quick classroom quiz

Use these as oral questions or a short written check.

What is the difference between append() and extend()?

Why does (10,) create a tuple but (10) does not?

What is the difference between dictionary keys and values?

How does a set help remove duplicates?

What does NumPy shape (2, 3) mean?

What is the output of np.array([2, 4]) + 3?

Which NumPy function calculates the mean?